PROCESS SYNCHRONIZATION

# A Comprehensive Comparison of Process Synchronization Mechanisms

Dekker • Peterson • Semaphore • Monitor • Readers–Writers

## Details

| | |
|---|---|
| **Title** | A Comprehensive Comparison of Process Synchronization Mechanisms |
| **Course** | Embedding System |
| **Author** | Najla Davarpanah |
| **Instructor** | Majid Shamshirgaran |
| **University / Faculty** | Eshragh University |
| **Date** | ۱۰ November ۲۰۲۶ |

English edition

# Abstract and Contents

When several processes access shared data concurrently, a correct synchronization mechanism is required to avoid inconsistency. This article compares five well-known approaches: Dekker's and Peterson's software algorithms for two processes, semaphores as an operating-system primitive, monitors as a language-level construct, and the classical readers–writers problem. Each is presented with pseudocode, C code, a flowchart, an execution trace, worked examples and exam-style questions. Comparison tables cover mutual exclusion, progress, bounded waiting, starvation, deadlock, busy waiting, scalability and more, and charts visualise the differences.

**Keywords:** synchronization, critical section, mutual exclusion, Dekker, Peterson, semaphore, monitor, readers–writers

## Contents


| | |
|---|---|
| ١ | Introduction and the Critical-Section Problem |
| ٢ | Dekker's Algorithm |
| ٣ | Peterson's Algorithm |
| ٤ | Semaphores |
| ٥ | Monitors |
| ٦ | Readers–Writers Problem |
| ٧ | Comprehensive Comparison Tables |
| ٨ | Charts |
| ٩ | Examples and Exam Questions |
| ١٠ | Conclusion |
| ١١ | References |

# ١. Introduction and the Critical-Section Problem

In multiprocess and multithreaded systems, processes read and write shared data to exchange information or use common resources. If concurrent access is not controlled, the result depends on the exact interleaving of instructions, a phenomenon called a **race condition**.

A classic example: processes `P1` and `P2` execute `counter++` and `counter--` respectively on a shared variable `counter = 5`. At machine level each statement takes three steps: load into a register, modify the register, store back to memory. Depending on how the steps interleave, the final value may be ٤, ٥ or ٦, although only ٥ is correct.

## Structure of a critical section

The part of the code that touches shared data is the **critical section (CS)**. Each process has four parts: entry, critical section, exit and remainder. The algorithms in this article are different ways of building the entry and exit sections.

**Pseudocode — general structure**

```
do {
    ENTRY SECTION        // ask for permission
        CRITICAL SECTION // touches shared data
    EXIT SECTION         // give permission to others
        REMAINDER SECTION
} while (true);
```

## The three correctness requirements

- **Mutual exclusion:** at most one process is inside the critical section at any time.
- **Progress:** if the critical section is free and some processes want to enter, only those processes take part in deciding who enters next, and the decision cannot be postponed indefinitely; a process in its remainder section must not block others.
- **Bounded waiting:** from the moment a process requests entry until it is granted, the number of times other processes enter first is bounded (no starvation).

Assumptions: nothing is assumed about the relative speed of processes, and reading or writing a simple variable is treated as an atomic action.

## Classification of the methods

**Table ١ — Classification of the methods**

| Method | Type | Implementation level | Requirements |
|---|---|---|---|
| **Dekker** | Software algorithm | User program | Shared memory only |
| **Peterson** | Software algorithm | User program | Shared memory only |
| **Semaphore** | Synchronization primitive | OS kernel / library | Atomic operations and a wait queue |
| **Monitor** | Structured construct | Language / library | Lock + condition variables |

| Method | Type | Implementation level | Requirements |
|---|---|---|---|
| **Readers–Writers** | Classical problem | Design pattern | Solved with semaphores or monitors |

> **Note**
> “Readers–writers” is a **problem**, not a primitive. In the comparison tables its column refers to the standard semaphore solution (reader priority); other solutions are noted separately.

> **In brief**
> A race condition arises when the result depends on the interleaving of operations on shared data. A correct solution must guarantee mutual exclusion, progress and bounded waiting without assuming anything about relative process speeds.

# ٢. Dekker's Algorithm

Dekker's algorithm is the first correct software solution to the critical-section problem for **two** processes. It was devised by the Dutch mathematician T. J. Dekker and published by Dijkstra in ١٩٦٥. It uses only shared memory and needs no special hardware instruction.

## Idea

There are two shared variables: the array `flag[2]`, which tells whether each process wants to enter, and `turn`, which decides priority in case of conflict. A process first announces its interest; if the rival is also interested and the turn is not its own, it **backs off** (lowers its flag), waits until it is its turn, and tries again. This back-off is what removes the deadlock of simpler schemes.

**Pseudocode — process Pi (j = ١ - i)**

```
shared: boolean flag[2] = {false, false};
        int turn = 0;

Pi:
  repeat
    flag[i] = true;
    while (flag[j] == true) {
        if (turn != i) {            // not my turn: back off
            flag[i] = false;
            while (turn != i) ;     // wait for my turn
            flag[i] = true;
        }
    }
    // ---- CRITICAL SECTION ----
    turn = j;                       // hand priority to the other
    flag[i] = false;
    // ---- REMAINDER SECTION ----
  forever
```

**C code (C١١ atomics)**

```
#include <stdatomic.h>
#include <stdbool.h>

atomic_bool flag[2] = { false, false };
atomic_int  turn    = 0;
```

```
void dekker_enter(int i) {
    int j = 1 - i;
    atomic_store(&flag[i], true);
    while (atomic_load(&flag[j])) {
        if (atomic_load(&turn) != i) {
            atomic_store(&flag[i], false);
            while (atomic_load(&turn) != i) { /* busy wait */ }
            atomic_store(&flag[i], true);
        }
    }
}

void dekker_exit(int i) {
    atomic_store(&turn, 1 - i);
    atomic_store(&flag[i], false);
}
```

The code uses atomic variables with the default `seq_cst` ordering. On modern processors, replacing them with ordinary variables lets memory reordering break the algorithm.

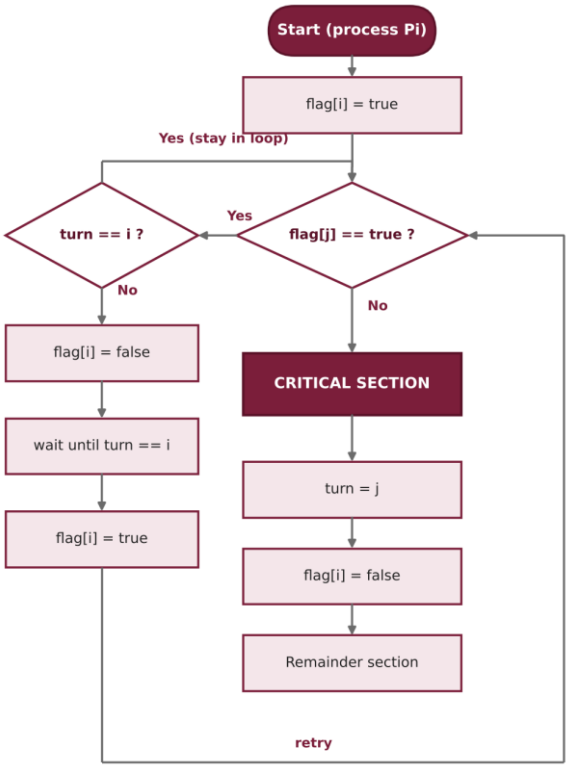



**Figure ١ — Flowchart of Dekker's algorithm for process Pi**

## Execution trace

Assume `turn = 0` and both processes want to enter at the same time:

**Table ٢ — Dekker trace (both processes request entry)**

| Step | P٠ | P١ | f[٠] | f[١] | turn |
|---|---|---|---|---|---|
| ١ | flag[٠]=true | flag[١]=true | T | T | ٠ |

| Step | P٠ | P١ | f[٠] | f[١] | turn |
|---|---|---|---|---|---|
| ٢ | flag[١] is true; turn==٠ → stays in loop | flag[٠] is true; turn≠١ → flag[١]=false | T | F | ٠ |
| ٣ | flag[١] is false → enters CS | waits for turn==١ | T | F | ٠ |
| ٤ | turn=١; flag[٠]=false (exit) | turn==١ → flag[١]=true | F | T | ١ |
| ٥ | remainder section | flag[٠] is false → enters CS | F | T | ١ |

## Properties

- **Mutual exclusion:** holds; entry is possible only when `flag[j]` is seen false.
- **Progress:** holds; if the rival is not interested, `flag[j]=false` and the process enters regardless of `turn`.
- **Bounded waiting:** holds; on exit `turn` is given to the rival, so each process waits for at most one entry of the rival.
- **Drawbacks:** only two processes; busy waiting; subtle logic; needs memory ordering on modern hardware.

> **In brief**
> Dekker's algorithm (١٩٦٥) uses flag[٢] to announce intent and turn to resolve conflicts; the process without priority backs off by lowering its flag. It guarantees mutual exclusion, progress and bounded waiting, but only for two processes and with busy waiting.

# ٣. Peterson's Algorithm

In ١٩٨١ G. L. Peterson published a much simpler two-process solution. The variables are the same (`flag[2]` and `turn`), but the logic is one of politeness: a process that wants to enter announces its interest and then **gives the turn to the rival**. If both want to enter at once, the one that wrote `turn` last has yielded and waits.

**Pseudocode — process Pi (j = ١ - i)**

```
shared: boolean flag[2] = {false, false};
        int turn;


Pi:
  repeat
    flag[i] = true;                     // I am interested
    turn = j;                           // but you go first
    while (flag[j] == true && turn == j) ;   // wait
    // ---- CRITICAL SECTION ----
    flag[i] = false;                    // I am done
    // ---- REMAINDER SECTION ----
  forever
```

**C code (C١١ atomics)**

```
#include <stdatomic.h>
#include <stdbool.h>


atomic_bool flag[2] = { false, false };
atomic_int  turn;


void peterson_enter(int i) {
    int j = 1 - i;
    atomic_store(&flag[i], true);
    atomic_store(&turn, j);
    while (atomic_load(&flag[j]) && atomic_load(&turn) == j) {
        /* busy wait */
    }
}


void peterson_exit(int i) {
    atomic_store(&flag[i], false);
}
```

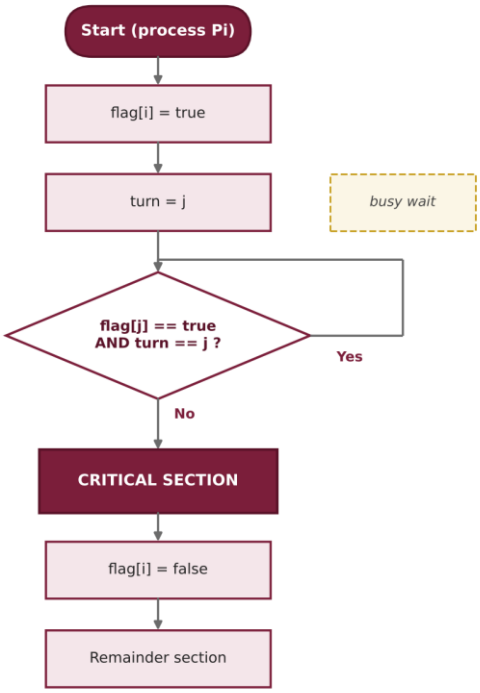



**Figure ٢ — Flowchart of Peterson's algorithm**

## Short proof of mutual exclusion

Suppose `P0` and `P1` are both in the critical section. Each saw the loop condition false before entering. `P0` saw either `flag[1]` false or `turn==0`; but `P1` had already written `flag[1]=true` and keeps it until exit, so `P0` must have seen `turn==0`. Symmetrically `P1` must have seen `turn==1`. Since `turn` is a single variable and the last writer decides, both cannot hold at once — a contradiction.

## Execution trace

**Table ٣ — Peterson trace (both processes request entry)**

| Step | P٠ | P١ | f[٠] | f[١] | turn |
|---|---|---|---|---|---|
| ١ | flag[٠]=true | — | T | F | ? |
| ٢ | — | flag[١]=true | T | T | ? |
| ٣ | turn=١ | — | T | T | ١ |
| ٤ | — | turn=٠ (last writer) | T | T | ٠ |
| ٥ | flag[١]&&turn==١? false → enters | flag[٠]&&turn==٠? true → waits | T | T | ٠ |
| ٦ | flag[٠]=false (exit) | condition false → enters | F | T | ٠ |

## Properties and notes

- **Mutual exclusion, progress and bounded waiting** all hold; the rival enters at most once before you.

- **Drawbacks:** only two processes, busy waiting, and it needs sequentially consistent memory; without `seq_cst` or a memory fence, the store buffer can reorder `flag[i]=true` and the read of `flag[j]`, letting both processes enter.
- Generalisation to `N` processes is possible with the **filter lock** or the Bakery algorithm, at higher cost and complexity.

> **In brief**
> Peterson's algorithm (١٩٨١) simplifies Dekker's idea: a process sets its flag and then yields the turn to the other. The last writer of turn waits. It satisfies all three requirements for two processes in a few lines, but needs sequentially consistent memory and busy waiting.

# ٤. Semaphores

Semaphores were introduced by Dijkstra in ١٩٦٥. A semaphore is an integer `S` modified only through two **atomic** operations: `wait` (Dijkstra's P, *proberen*) and `signal` (V, *verhogen*). Unlike Dekker and Peterson, semaphores work for **N processes**, and in the blocking implementation a waiting process is put to sleep in a queue instead of spinning, which frees the processor.

## Definition and blocking implementation

**Pseudocode — semaphore operations**

```
typedef struct { int value; queue list; } semaphore;

wait(S):                      // P, down  (executed atomically)
    S.value = S.value - 1;
    if (S.value < 0) {
        add this process to S.list;
        block();              // sleep: no busy waiting
    }

signal(S):                    // V, up    (executed atomically)
    S.value = S.value + 1;
    if (S.value <= 0) {
        remove a process P from S.list;
        wakeup(P);
    }
```

In this implementation a negative value of `S` equals the number of waiting processes. Two common kinds exist: the **binary semaphore (mutex)** with value ٠ or ١ for mutual exclusion, and the **counting semaphore** for multi-instance resources.

**Pseudocode — mutual exclusion**

```
semaphore mutex = 1;

Pi:
  repeat
    wait(mutex);
      // ---- CRITICAL SECTION ----
    signal(mutex);
      // ---- REMAINDER SECTION ----
  forever
```

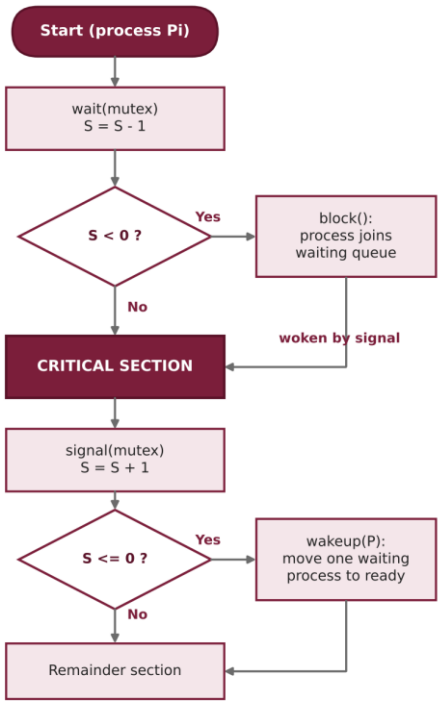



**Figure ٣ — Flowchart of semaphore usage (wait / signal)**

## Example: bounded-buffer producer–consumer

Three semaphores are needed: `empty` (free slots, initially `N`), `full` (filled slots, initially ٠) and `mutex` (exclusive access to the buffer).

**Pseudocode**

```
semaphore mutex = 1, empty = N, full = 0;

Producer:                          Consumer:
  repeat                             repeat
    item = produce();                  wait(full);
    wait(empty);                       wait(mutex);
    wait(mutex);                       item = remove();
    insert(item);                      signal(mutex);
    signal(mutex);                     signal(empty);
    signal(full);                      consume(item);
  forever                            forever
```

**C code (POSIX threads + semaphores)**

```
#include <pthread.h>
#include <semaphore.h>
#define N 8

int buf[N], in = 0, out = 0;
sem_t empty, full, mutex;

void *producer(void *arg) {
    for (int i = 0; i < 100; i++) {
        sem_wait(&empty);
        sem_wait(&mutex);
        buf[in] = i;  in = (in + 1) % N;
```

```
        sem_post(&mutex);
        sem_post(&full);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 100; i++) {
        sem_wait(&full);
        sem_wait(&mutex);
        int x = buf[out];  out = (out + 1) % N;
        sem_post(&mutex);
        sem_post(&empty);
        (void)x;                /* use the item here */
    }
    return NULL;
}

int main(void) {
    pthread_t p, c;
    sem_init(&empty, 0, N);
    sem_init(&full,  0, 0);
    sem_init(&mutex, 0, 1);
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    return 0;
}
```

## Properties and pitfalls

- **Mutual exclusion and progress** hold with correct use. **Bounded waiting** depends on the queue policy: a strong semaphore (FIFO queue) guarantees it, a weak one (arbitrary wake-up) does not.
- **Deadlock:** a wrong order of operations, e.g. `wait(mutex)` before `wait(empty)` in the producer, can deadlock.
- **Error-prone:** forgetting a `signal` or swapping `wait` and `signal` breaks mutual exclusion or blocks processes forever; it is an unstructured, low-level tool.
- **Priority inversion** may occur and calls for protocols such as priority inheritance.

> **In brief**
> A semaphore is an integer manipulated only by atomic wait (P) and signal (V). Blocking semaphores avoid busy waiting and work for any number of processes, but are low-level and error-prone. Bounded waiting holds only with FIFO (strong) semaphores.

# ٥. Monitors

Monitors were proposed by C. A. R. Hoare (١٩٧٤) and Brinch Hansen to reduce the error-proneness of semaphores. A monitor is an **abstract data type** that holds shared data and the procedures that access it, and the **language or compiler** guarantees that at most one process is active inside its procedures at any time, so the programmer does not write `wait/signal` for mutual exclusion.

For conditional synchronization (e.g. “the buffer is not empty”) a monitor uses **condition variables** with two operations: `x.wait()` blocks the process and releases the monitor lock, and `x.signal()` wakes one waiting process. Unlike a semaphore `signal`, a condition `signal` with no waiter has **no effect** and is not remembered.

## Two signal semantics

- **Hoare (signal-and-wait):** the signalling process hands the monitor to the woken one immediately; the condition is guaranteed to hold, so `if` suffices.
- **Mesa (signal-and-continue):** the signaller continues and the woken process re-acquires the lock later; the condition may have changed, so a **`while`** loop is required. Java, pthreads and most real systems follow Mesa.

**Pseudocode — bounded-buffer monitor**

```
monitor BoundedBuffer {
    int buf[N];  int count = 0, in = 0, out = 0;
    condition notFull, notEmpty;

    procedure put(x) {
        while (count == N) notFull.wait();
        buf[in] = x;  in = (in + 1) % N;  count++;
        notEmpty.signal();
    }

    procedure get() {
        while (count == 0) notEmpty.wait();
        x = buf[out]; out = (out + 1) % N; count--;
        notFull.signal();
        return x;
    }
}   // at most one process is active inside at any time
```

**C code (pthread mutex + condition variables)**

```
#include <pthread.h>
#define N 8

static int buf[N], in = 0, out = 0, count = 0;
static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;  /* monitor lock */
static pthread_cond_t notFull  = PTHREAD_COND_INITIALIZER;
static pthread_cond_t notEmpty = PTHREAD_COND_INITIALIZER;

void put(int x) {
    pthread_mutex_lock(&m);                      /* enter monitor */
    while (count == N) pthread_cond_wait(&notFull, &m);
    buf[in] = x;  in = (in + 1) % N;  count++;
    pthread_cond_signal(&notEmpty);
    pthread_mutex_unlock(&m);                    /* exit monitor  */
}
```

```
int get(void) {
    pthread_mutex_lock(&m);
    while (count == 0) pthread_cond_wait(&notEmpty, &m);
    int x = buf[out];  out = (out + 1) % N;  count--;
    pthread_cond_signal(&notFull);
    pthread_mutex_unlock(&m);
    return x;
}
```

In Java the same pattern is written with `synchronized` and `wait()/notifyAll()`. C has no built-in monitor; here it is **emulated** with a lock and condition variables.

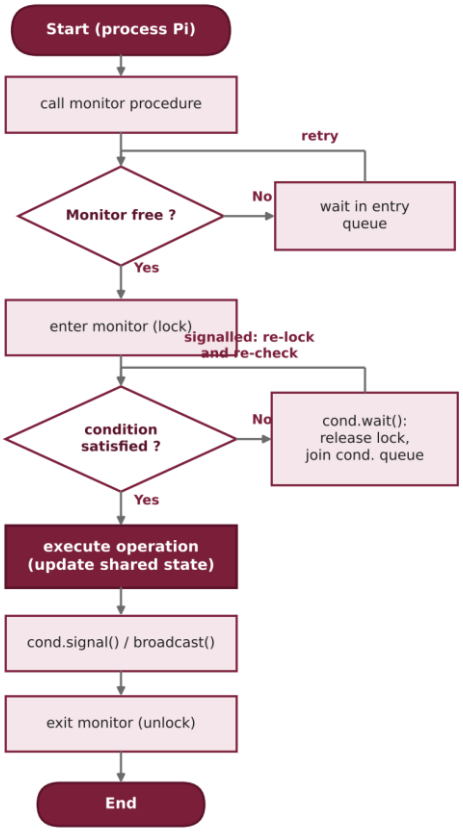



**Figure ٤ — Flowchart of monitor entry/exit with a condition variable**

## Properties and pitfalls

- **Automatic, structured mutual exclusion**; the safest and most readable tool in this article.
- **Deadlock** is still possible, e.g. through nested monitor calls that hold another lock.
- **Starvation** is possible under Mesa semantics if woken processes repeatedly find the condition false and the queue is not fair.
- Needs language or library support; has lock and context-switch costs.

> **In brief**
> A monitor bundles shared data and its procedures; the language guarantees that only one process is active inside at a time. Condition variables provide conditional synchronization; a signal with no waiter is lost. With Mesa semantics, always re-check the condition in a while loop.

# ٦. Readers–Writers Problem

The readers–writers problem was posed by Courtois, Heymans and Parnas in ١٩٧١. A shared data object (e.g. a database) is used by **readers**, which only read and may run concurrently, and **writers**, which need fully exclusive access: no other writer and no reader may be active together with a writer. Unlike plain mutual exclusion, concurrency among readers is allowed and raises throughput.

## Three variants

**Table ٤ — Variants of the readers–writers problem**

| Variant | Rule | Consequence |
|---|---|---|
| **First — reader priority** | While some reader is reading, a new reader may enter | Possible **writer starvation** |
| **Second — writer priority** | When a writer is waiting, no new reader may enter | Possible **reader starvation** |
| **Third — fair** | Entry in arrival order (FIFO) for all requests | No starvation; less concurrency |

## Semaphore solution (reader priority)

`rw_mutex` controls access to the data, `read_count` holds the number of active readers, and `mutex` protects `read_count`. The first reader locks writers out and the last reader lets them in again.

**Pseudocode**

```
semaphore rw_mutex = 1, mutex = 1;
int read_count = 0;

Writer:                          Reader:
  wait(rw_mutex);                  wait(mutex);
    // WRITE                       read_count++;
  signal(rw_mutex);                if (read_count == 1) wait(rw_mutex);
                                   signal(mutex);
                                     // READ
                                   wait(mutex);
                                   read_count--;
                                   if (read_count == 0) signal(rw_mutex);
                                   signal(mutex);
```

**C code (POSIX semaphores)**

```
#include <semaphore.h>

sem_t rw_mutex, mutex;     /* both initialised to 1 */
int   read_count = 0;

void writer(void) {
    sem_wait(&rw_mutex);
    /* ... write shared data ... */
    sem_post(&rw_mutex);
}

void reader(void) {
    sem_wait(&mutex);
    if (++read_count == 1) sem_wait(&rw_mutex);   /* first reader */
    sem_post(&mutex);
```

```
    /* ... read shared data ... */

    sem_wait(&mutex);
    if (--read_count == 0) sem_post(&rw_mutex);   /* last reader  */
    sem_post(&mutex);
}
```

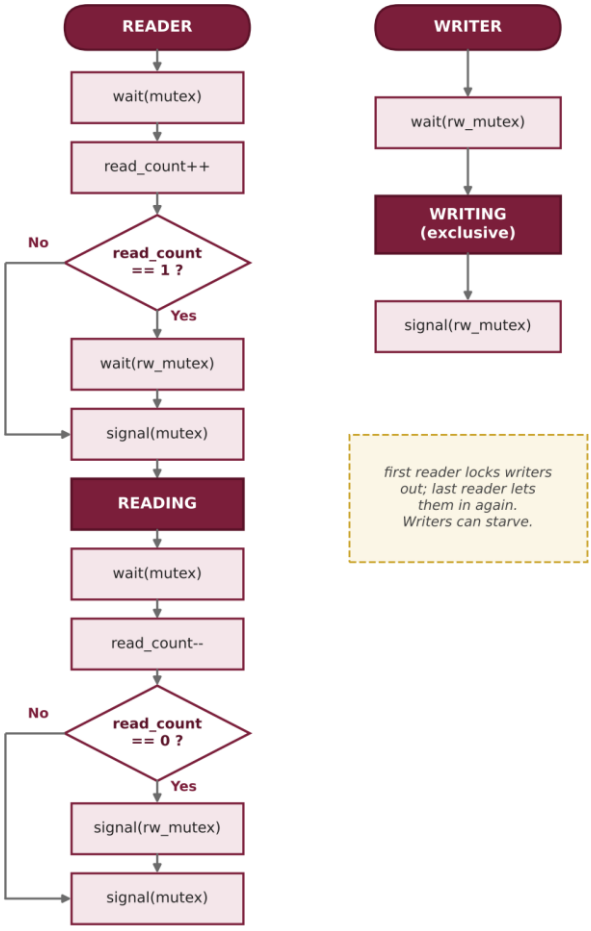



**Figure ٥ — Flowchart of reader and writer (semaphore solution)**

## Fair version with semaphores

Adding a queue semaphore `service` that every request passes through preserves arrival order:

**Pseudocode — third problem**

```
semaphore service = 1, rw_mutex = 1, mutex = 1;  int read_count = 0;

Reader:                              Writer:
  wait(service);                       wait(service);
  wait(mutex);                         wait(rw_mutex);
  if (++read_count == 1)               signal(service);
      wait(rw_mutex);                    // WRITE
  signal(service);                     signal(rw_mutex);
  signal(mutex);
    // READ
  wait(mutex);
  if (--read_count == 0) signal(rw_mutex);
  signal(mutex);
```

## Monitor solution (writer priority)

**Pseudocode**

```
monitor ReadersWriters {
    int readers = 0, waitingWriters = 0;  boolean writing = false;
    condition okToRead, okToWrite;

    procedure startRead() {
        while (writing || waitingWriters > 0) okToRead.wait();
        readers++;
    }
    procedure endRead() {
        readers--;
        if (readers == 0) okToWrite.signal();
    }
    procedure startWrite() {
        waitingWriters++;
        while (writing || readers > 0) okToWrite.wait();
        waitingWriters--;  writing = true;
    }
    procedure endWrite() {
        writing = false;
        if (waitingWriters > 0) okToWrite.signal();
        else okToRead.broadcast();
    }
}
```

**C code (pthread monitor)**

```c
#include <pthread.h>

static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t okToRead  = PTHREAD_COND_INITIALIZER;
static pthread_cond_t okToWrite = PTHREAD_COND_INITIALIZER;
static int readers = 0, writing = 0, waitingWriters = 0;

void start_read(void) {
    pthread_mutex_lock(&m);
    while (writing || waitingWriters > 0) pthread_cond_wait(&okToRead, &m);
    readers++;
    pthread_mutex_unlock(&m);
}
void end_read(void) {
    pthread_mutex_lock(&m);
    if (--readers == 0) pthread_cond_signal(&okToWrite);
    pthread_mutex_unlock(&m);
}
void start_write(void) {
    pthread_mutex_lock(&m);
    waitingWriters++;
    while (writing || readers > 0) pthread_cond_wait(&okToWrite, &m);
    waitingWriters--;  writing = 1;
    pthread_mutex_unlock(&m);
}
void end_write(void) {
    pthread_mutex_lock(&m);
    writing = 0;
    if (waitingWriters > 0) pthread_cond_signal(&okToWrite);
    else pthread_cond_broadcast(&okToRead);
    pthread_mutex_unlock(&m);
}
```

POSIX also provides a ready-made reader–writer lock (`pthread_rwlock_t`), which is preferable to hand-written code in practice.

**In brief**

Readers may share the data; a writer needs exclusive access. The first solution (reader priority) can starve writers, the second (writer priority) can starve readers, and the third uses a service queue to be fair. Both semaphores and monitors can implement all three.

# ٧. Comprehensive Comparison Tables

This section compares the five methods on correctness parameters (Table ٥) and engineering parameters (Table ٦). The “Readers–Writers” column refers to the standard semaphore solution (reader priority) unless stated otherwise.

**Table ٥ — Correctness and behaviour**

| Parameter | Dekker | Peterson | Semaphore | Monitor | Readers–Writers |
|---|---|---|---|---|---|
| **Mutual exclusion** | ✓ | ✓ | ✓(if used correctly) | ✓(automatic) | ✓(writer vs. all) |
| **Progress** | ✓ | ✓ | ✓ | ✓ | ✓ |
| **Bounded waiting** | ✓(at most once) | ✓(at most once) | Only with FIFO queue | Usually with FIFO queue | Variant ١ ✗ variant ٣ |
| **Starvation** | None | None | Possible (weak semaphore) | Possible (Mesa) | Writer (var. ١) / reader (var. ٢) |
| **Deadlock** | None | None | Possible with wrong order | Possible (nested calls) | Possible with wrong order |
| **Busy waiting** | Yes | Yes | No (blocking type) | No | No |
| **Fairness** | Nearly fair (٢ procs) | Fair (٢ procs) | Depends on queue | Depends on cond. queue | Variant ٣ is fair |
| **Reader concurrency** | — | — | — | — | Yes (goal of the problem) |

**Table ٦ — Engineering and practical aspects**

| Parameter | Dekker | Peterson | Semaphore | Monitor | Readers–Writers |
|---|---|---|---|---|---|
| **Year / inventor** | ١٩٦٥, Dekker / Dijkstra | ١٩٨١, Peterson | ١٩٦٥, Dijkstra | ١٩٧٤, Hoare / Brinch Hansen | ١٩٧١, Courtois et al. |
| **Type** | Software algorithm | Software algorithm | OS primitive | Language construct | Classical problem |
| **Number of processes** | ٢ | ٢ (generalised: Filter) | N | N | N readers + N writers |
| **Hardware / OS need** | Memory only (+ memory ordering) | Memory only (+ memory ordering) | Atomic ops + OS queue | Lock + condition variable | Semaphore or monitor |
| **Shared variables** | ٣ | ٣ | ١ (mutex) | ٠ explicit (internal lock) | ٣ (rw_mutex, mutex, read_count) |
| **Abstraction level** | Low | Low | Medium | High | Design pattern |
| **Ease of use** | Hard | Fairly simple | Medium | Easy | Medium |
| **Error-proneness** | High | Medium | High | Low | Medium |
| **Run-time cost** | High CPU use | High CPU use | Context switch when waiting | Lock + context switch | Same as base tool |
| **Typical use** | Teaching, theory | Teaching, theory, simple systems | Kernels, threads, counted resources | Java, C#, libraries | Databases, caches |

**Summary of the tables**

Dekker and Peterson suit **teaching** and the theory of correctness; semaphores suit **more than two processes** without busy waiting; monitors give **safer and more readable code**; and the readers–writers pattern fits data with many readers and few writers, with the policy (reader, writer or fair) chosen to match the workload.

# ٨. Charts

The charts below visualise several aspects of the comparison. The scores in Figures ٦ and ٧ are the **author's qualitative assessment** based on structural properties of each method (١ = weak, ٥ = strong), not the result of empirical measurements.

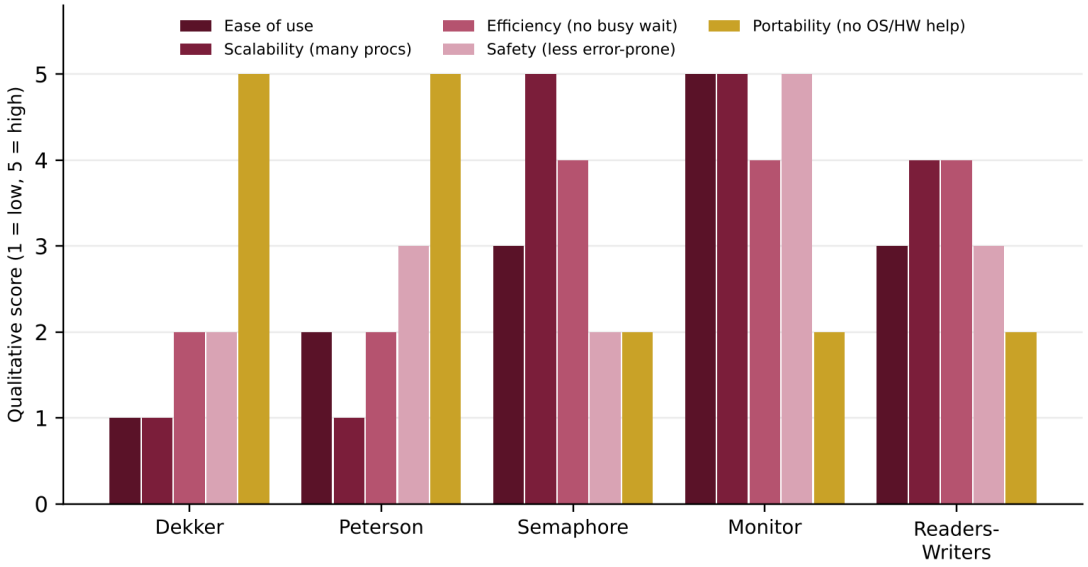



**Figure ٦ — Qualitative comparison on five criteria (bar chart)**




**Figure ٧ — Radar chart of the same criteria**

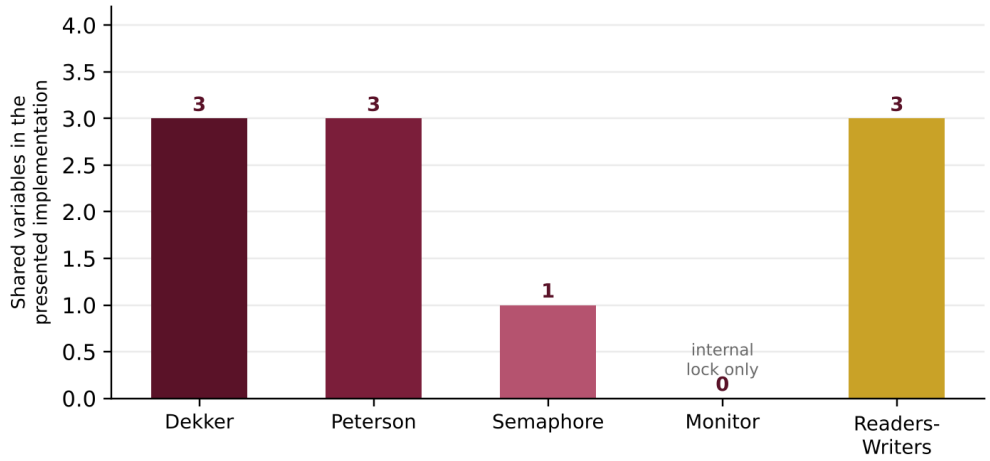



**Figure ٨ — Shared variables in the presented implementations**

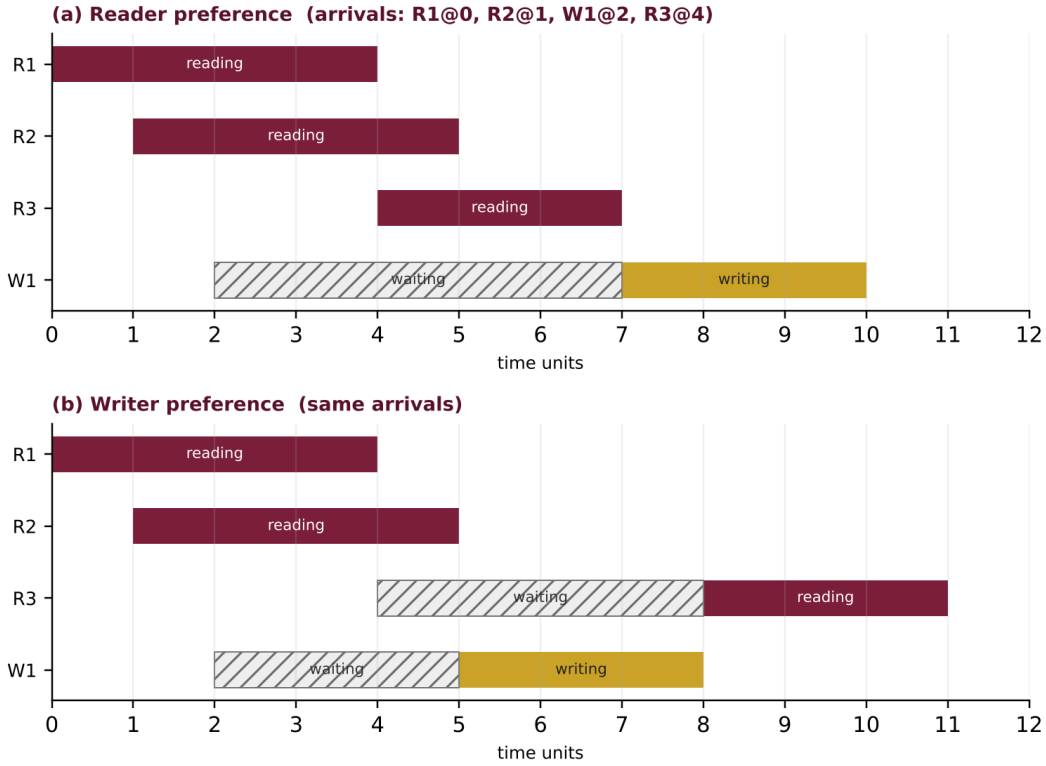



**Figure ٩ — Sample timeline: reader preference vs. writer preference**

In Figure ٩ (an illustrative scenario, not a measurement) `R1` arrives at time ٠, `R2` at ١, `W1` at ٢ and `R3` at ٤. With reader priority `R3` is admitted because readers are already reading, and `W1` waits until time ٧. With writer priority `R3` is held behind `W1`, and `W1` writes once `R1` and `R2` finish (time ٥).

# ۹. Examples and Exam Questions

For each of the five methods, one worked example and several questions with answers are given.

## ۹,۱ Dekker

> **Worked example**
> If `turn = 1` and only `P0` wants to enter, what happens? `P0` writes `flag[0]=true`, sees `flag[1]` false and enters **immediately**, even though the turn belongs to `P1`. This is the progress property: `turn` matters only when there is a conflict.

**Question ۱:** What happens if the statement `flag[i] = false` inside the loop is removed from Dekker's algorithm?

> **Answer:** Deadlock. Suppose `turn = 0` and both flags are true. `P1`, since `turn ≠ 1`, waits for `turn == 1` but keeps its flag true; `P0` sees `flag[1]` true and spins in the `while` loop. `turn` changes only after `P0` exits, so neither makes progress.

**Question ۲:** For how many processes does Dekker's algorithm work, and does it have bounded waiting?

> **Answer:** Only ۲ processes. Yes, it has bounded waiting: after exit, `turn` is given to the rival, so each process waits for at most one entry of the rival.

## ۹,۲ Peterson

> **Worked example**
> If `P1` is in its remainder section, `flag[1]=false`. `P0` writes its flag and `turn=1`, but the condition `flag[1] && turn==1` is false, so it enters: a lone requester is never blocked.

**Question ۳:** If the two entry statements are swapped (first `turn = j`, then `flag[i] = true`), is mutual exclusion violated? Build a scenario.

> **Answer:** Yes. `P0: turn=1`; `P1: turn=0`; `P1: flag[1]=true`; `P1` sees `flag[0]` false and enters. Then `P0: flag[0]=true`; the condition `flag[1] && turn==1` is false because `turn==0`, so `P0` enters too, and both are inside the critical section.

**Question ٤:** Why can Peterson's algorithm fail on modern processors without a memory fence?

> **Answer:** The processor or compiler may reorder the write of `flag[i]=true` and the read of `flag[j]` (especially with a store buffer). Both processes may then read the other's flag as false and enter. The fix is `seq_cst` atomics or a memory fence.

## ٩,٣ Semaphore

**Worked example — counting semaphore with initial value ٢**

Four processes `A..D` execute `wait(S)` in order, then `A` and `B` execute `signal(S)`:

`A: S=1`; `B: S=0`; `C: S=-1` (blocked); `D: S=-2` (blocked); `A: signal → S=-1`, `C` wakes; `B: signal → S=0`, `D` wakes. A negative value shows the number of waiters.

**Question ٥:** Using a semaphore, guarantee that statement `S1` of `P1` always runs before `S2` of `P2`.

> **Answer:** Create semaphore `sync` with value ٠. `P1: S1; signal(sync);` and `P2: wait(sync); S2;`. If `P2` arrives first it blocks until `P1` signals.

**Question ٦:** Show how a wrong order of `wait` calls creates a deadlock.

> **Answer:** With two semaphores `A=1` and `B=1`: `P1: wait(A); wait(B);` and `P2: wait(B); wait(A);`. If `P1` takes `A` and `P2` takes `B`, each waits for the other forever. Fix: always acquire semaphores in a fixed global order.

## ٩,٤ Monitor

**Worked example — buffer of capacity ٢**

`put(x)` and `put(y)` fill the buffer (`count=2`). `put(z)` blocks in `notFull.wait()` and releases the lock. `get()` removes an item, `count=1`, and `notFull.signal()` wakes the waiting producer; it re-checks `count==N`, finds it false and inserts `z`.

**Question ٧:** Why must a Mesa-style monitor wait on a condition with `while` rather than `if`?

> **Answer:** Between the `signal` and the re-acquisition of the lock, another process may enter and change the condition (or a spurious wake-up may occur). `while` re-checks the condition after waking.

**Question ٨:** What is the difference between a semaphore `signal` and a condition-variable `signal`?

> **Answer:** A semaphore `signal` increments the counter and is **remembered**; a condition `signal` with no waiter has **no effect** and is lost.

## ٩,٥ Readers–Writers

**Worked example — reader priority**

`R1` enters (`read_count=1`, takes `rw_mutex`). `W1` blocks on `wait(rw_mutex)`. `R2` arrives: `read_count=2`, not ١, so it does not need `rw_mutex` and enters. As long as readers keep arriving one after another, `read_count` never reaches ٠ and `W1` keeps waiting — writer starvation.

**Question ٩:** What are the roles of `mutex` and `rw_mutex` in the semaphore solution?

> **Answer:** `mutex` only protects the variable `read_count` from concurrent updates by readers. `rw_mutex` controls access to the data: a writer takes it fully, while readers take it as a group (the first reader acquires it, the last releases it).

**Question ١٠:** Which variants suffer from starvation, and how can it be fixed?

> **Answer:** Variant ١ starves writers and variant ٢ starves readers. Variant ٣ uses a service queue (a `service` semaphore, or a FIFO queue in a monitor) to preserve arrival order and has no starvation.

**Question ١١:** (Multiple choice) Which method works for more than two processes and has no busy waiting? a) Dekker b) Peterson c) blocking semaphore d) none

> **Answer:** Option **c**; Dekker and Peterson support only two processes and use busy waiting.

# ١٠. Conclusion

Dekker's and Peterson's algorithms show that mutual exclusion can be built with shared memory alone and no special instruction, but they are limited to two processes, use busy waiting and depend on correct memory ordering. Semaphores are scalable and efficient but low-level and error-prone. Monitors add structure and automatic mutual exclusion, making concurrent programming safer, and are very common in modern languages. The readers–writers problem shows that choosing the **policy** (reader priority, writer priority or fairness) matters as much as choosing the primitive.

- To learn correctness concepts: Dekker and Peterson.
- To control resources and execution order among many processes: semaphores.
- For structured, low-error code: monitors (or lock + condition variable).
- For shared data that is mostly read: the readers–writers pattern or `pthread_rwlock_t`.

> **Summary**
> Pick the primitive by the number of processes and the level of safety needed, and then pick the policy that matches the workload.

# ١١. References


- Dijkstra, E. W. (١٩٦٥). Cooperating Sequential Processes (EWD١٢٣). Technological University Eindhoven.
- Peterson, G. L. (١٩٨١). Myths About the Mutual Exclusion Problem. Information Processing Letters, ١٢(٣), ١١٥–١١٦.
- Hoare, C. A. R. (١٩٧٤). Monitors: An Operating System Structuring Concept. Communications of the ACM, ١٧(١٠), ٥٤٩–٥٥٧.
- Brinch Hansen, P. (١٩٧٣). Operating System Principles. Prentice Hall.
- Courtois, P. J., Heymans, F., & Parnas, D. L. (١٩٧١). Concurrent Control with “Readers” and “Writers”. Communications of the ACM, ١٤(١٠), ٦٦٧–٦٦٨.
- Silberschatz, A., Galvin, P. B., & Gagne, G. Operating System Concepts. Wiley.
- Tanenbaum, A. S., & Bos, H. Modern Operating Systems. Pearson.